

DESIGN PATTERNS IN PRACTICE

A Portfolio of Architecture Patterns Across Enterprise Banking Platforms

Eleven proven patterns, mapped to real projects delivered

Placida Fernando
Senior Solution Architect | 25+ Years in Regulated Financial Services

Design Pattern Reference Table

Design Pattern	Where It Was Used
Gatekeeper	Soft Credit Pull microservice, where an internal API Gateway enforced OAuth2 auth before requests reached backend services
Federated Identity	Soft Credit Pull microservice, where trust was delegated to external identity providers (PingID) via OAuth2 Client Credentials flow
Backends for Frontends	Channel-specific API layer, using Node.js "outer" BFF APIs to decouple channels from core Java microservices
Strangler Fig	Core banking platform modernization , where COBOL/DB2 was incrementally migrated through SOAP/ESB to Java microservices/REST
Saga	Event-driven disbursement service, coordinating multi-step fund movement from loan approval to account credit, with compensating steps
Orchestration vs. Choreography	Event-driven disbursement service , choreographed via Kafka events rather than a central orchestrator
Publisher/Subscriber	Event-driven disbursement service, using a Kafka event bus to decouple publishers from subscribers
CQRS	Real-time derivatives reporting platform, separating the reporting/read workload from transactional capital markets systems
Gateway	Real-time derivatives reporting platform, using IBM WebSphere MQ as a single integration entry point, replacing point-to-point links
Facade	Retail banking middleware platform, presenting one unified service layer in front of core banking systems, databases, and channels
Sidecar	Kubernetes/OpenShift deployment , running Dynatrace instrumentation as a companion process alongside each service

Soft Credit Pull Microservice

- Lead architect for a reusable, builder-pattern microservice adopted enterprise-wide across HELOC, FlexPay, Unsecured Installment Loans, and Auto Lending, saving the bank millions through reuse.
- Integrated with FICO, MDM, and identity-verification vendors including eCBSV, Crosscore/FraudNet, PreciseID, MiTek, PingID, Inscribe.ai, and The Work Number.
- Designed service-to-service authentication using OAuth2 Client Credentials flow with Spring Security, enforced through an internal API Gateway before any request reached a backend service.

DESIGN PATTERNS

GK

FI

Gatekeeper + Federated Identity

The internal API Gateway centralized authentication, authorization, and request validation before traffic ever touched a backend node - a textbook Gatekeeper. Trust for who the caller is was delegated outward to identity providers like PingID, so services never managed credentials themselves - Federated Identity.

Channel-Specific API Layer

- Built "outer" Backend-for-Frontend APIs in Node.js sitting in front of core Java Spring Boot microservices.
- Decoupled channel-specific front-end consumption (web, mobile, partner channels) from the shape and pace of change of core lending services.
- Allowed each channel team to evolve its API contract independently, without forcing changes onto the shared core services.

DESIGN PATTERN



BFF

Backends for Frontends

Each frontend got its own tailored service layer instead of sharing one general-purpose API. This narrowed each surface to exactly what that channel needed, simplified security and authorization per channel, and let core microservices stay stable while channel experiences changed underneath.

Core Banking Platform Modernization

- Led legacy modernization of homegrown core banking platform, originally built on COBOL/DB2 with IBM WebSphere MQ.
- Migrated functionality first to a SOAP/ESB layer as an intermediate step, then incrementally to a Java microservices/REST layer.
- Kept IBM MQ in place throughout to integrate data with core banking systems, so downstream consumers were never broken mid-migration.

DESIGN PATTERN

SF

Strangler Fig

Rather than a risky big-bang rewrite, new microservices grew up around the COBOL/DB2 core one capability at a time - first behind SOAP/ESB, then REST - gradually taking over traffic until the legacy platform's footprint was “strangled” down to what still needed it.

Event-Driven Disbursement Service

- Designed the disbursement service using RTP network, ACH, and Kafka to move funds from loan approval to client account same day with minimal manual intervention.
- Integrated core banking and lending origination systems as independent participants reacting to events, rather than one controller directing every step.
- Each stage - approval, funds movement, account credit - published and consumed events, with compensating steps available if a downstream stage failed.

DESIGN PATTERNS



Saga (Choreographed) + Pub/Sub

The end-to-end disbursement is a long-running transaction spanning multiple systems - a Saga - coordinated through Choreography: each service reacts to events rather than waiting on a central orchestrator. Kafka served as the event bus, decoupling publishers from subscribers and adding a natural segmentation boundary.

Real-Time Derivatives Reporting Platform

- Architected a real-time derivatives reporting middleware platform using IBM WebSphere MQ messaging.
- Eliminated point-to-point integrations across capital markets systems by routing all integration traffic through a shared messaging layer.
- Kept the reporting/query workload physically separate from the transactional capital-markets systems feeding it, so heavy reporting queries never competed with trading throughput.

DESIGN PATTERNS



CQRS + Gateway

Separating the read-heavy reporting platform from the write-heavy trading systems is the essence of CQRS: reads and writes evolve and scale independently. WebSphere MQ acted as the Gateway - a single, well-governed entry point that replaced a tangle of point-to-point capital-markets integrations.

Retail Banking Middleware Platform

- Designed and developed a service-based retail banking middleware platform in Java J2EE/Spring, including Swing-based client interfaces deployed on Apache HTTP Server.
- Integrated core banking systems, databases, and multiple customer channels behind one consistent service layer.
- Provided technical support for bank's global payment exchange systems, including NACHA, ACH, and Real-Time Payments.

DESIGN PATTERN

FC

Facade

Client applications and downstream channels never spoke to core banking systems, databases, and payment networks directly. The middleware platform presented one simplified, consistent interface in front of that complexity - a classic Facade that let channel teams build without needing to understand every system behind it.

Containerized Deployment & Observability

- Deployed containerized Spring Boot services to Kubernetes, monitored via OpenShift, with externalized configuration and CI/CD pipelines on Azure.
- Instrumented services for Dynatrace so telemetry - logs, traces, and metrics - was captured without embedding monitoring logic inside application code.
- Aligned deployment and configuration practices with 12-factor app principles across the lending platform.

DESIGN PATTERN

SC

Sidecar

Observability was attached as a companion process alongside each service rather than coded into it - the same pod, a separate concern. That kept cross-cutting telemetry and monitoring logic out of business code, and let the observability layer be upgraded independently of the services it watched.

Modern Patterns, Applied Across a my Career



Gatekeeper

Soft Credit Pull



Federated Identity

Soft Credit Pull



Backends for Frontends

Channel API layer



Strangler Fig

Core banking modernization



Saga

Disbursement service



Orchestration vs. Choreography

Disbursement service



Publisher/Subscriber

Disbursement service



CQRS

Derivatives reporting



Gateway

Derivatives reporting



Facade

Retail banking middleware



Sidecar

Kubernetes/OpenShift